Configuração centralizada com Spring Cloud Config

RELATÓRIO DO TRABALHO PRÁTICO | 13/10/2026

Integrantes: Fernando, Felipe Montalvão e Victor Hugo

1. Arquitetura e conexão dos serviços

O projeto contém quatro aplicações independentes, geradas pelo Spring Initializr, com Java 21, Spring Boot 4.0.8 e Spring Cloud 2025.1.3. O Config Server utiliza @EnableConfigServer para fornecer configurações armazenadas em Git. Os clientes usam Spring Web, Actuator e Config Client; o BOM do Spring Cloud controla as versões das dependências.

Aplicação	Porta	Configuração selecionada
config-server	8888	Repositório Git informado em CONFIG_GIT_URI
cliente-vendas	8081	cliente-vendas-{dev ou prod}.properties
cliente-estoque	8082	cliente-estoque-{dev ou prod}.properties
cliente-relatorios	8083	cliente-relatorios-{dev ou prod}.properties

Cada cliente consulta o servidor usando o nome da aplicação, o perfil ativo e a branch main. As propriedades remotas definem a mensagem, a porta e o ambiente. O endpoint /mensagem retorna o texto injetado por @Value; /configuracao retorna serviço, ambiente e mensagem em JSON.

2. Propriedades do Config Server

Propriedade	Função
spring.application.name / server.port	Identifica config-server e define a porta 8888.
spring.cloud.config.server.git.uri	URI do Git, configurável por CONFIG_GIT_URI. Na entrega, usar a URL real do GitHub.
spring.cloud.config.server.git.clone-on-start	true: clona e valida o repositório na inicialização.
spring.cloud.config.server.git.default-label	Seleciona main; CONFIG_GIT_LABEL permite alterar a branch.
spring.cloud.config.server.git.try-master-branch	false: não procura automaticamente a branch master.
spring.cloud.config.server.git.refresh-rate	0: consulta atualizações no Git a cada requisição.
management.endpoints.web.exposure.include	*: expõe os endpoints Actuator exigidos para a demonstração.
server.address	127.0.0.1: execução dos serviços na máquina local.

Conexão, fluxo e validação

3. Propriedades dos clientes

Propriedade	Função
spring.application.name	Nome usado para localizar o arquivo de cada cliente.
spring.profiles.active	dev por padrão; APP_PROFILE ou argumento de execução permite selecionar prod.
spring.config.import	optional:configserver:http://localhost:8888; importa as propriedades do servidor. CONFIG_SERVER_URL permite mudar a URL.
spring.cloud.config.label	main por padrão; seleciona a versão/branch consultada.
server.port / server.address	Portas locais de fallback e acesso limitado a localhost.
management.endpoints.web.exposure.include	*: disponibiliza Actuator, incluindo health e refresh.
mensagem.boasvindas / app.ambiente	Valores remotos exibidos nos endpoints dos clientes.

4. Inicialização e atualização

Fluxo: iniciar o Config Server; ele acessa o Git e fica disponível na porta 8888. Iniciar os três clientes; cada um consulta o servidor, recebe as propriedades do seu perfil e passa a responder na sua porta. A importação pode consultar primeiro o perfil default e depois o perfil ativo.

Para demonstrar PROD, parar os clientes DEV e reiniciá-los com --spring.profiles.active=prod. Para atualizar uma mensagem: alterar o arquivo no Git, fazer commit e push, consultar o servidor e executar POST /actuator/refresh com conteúdo JSON no cliente. O @RefreshScope renova o controlador sem reiniciar o processo. O commit sozinho não atualiza os clientes; mudanças de porta exigem reinicialização.

5. Testes realizados e resultados

Compilação Maven concluída com BUILD SUCCESS nas quatro aplicações. A suíte de integração executou os JARs reais e aprovou 30 verificações, registradas em docs/evidencias-testes.json.

Consulta / cenário	Resultado
localhost:8888/actuator/health	UP; Config Server disponível.
localhost:8888/cliente-{vendas,estoque,relatorios}/{dev,prod}	Mensagens, portas, ambientes e SHA do Git corretos.
localhost:{8081,8082,8083}/mensagem e /configuracao	Conteúdo correto para os três clientes nos dois perfis.
Novo commit + POST localhost:8081/actuator/refresh	Nova mensagem carregada; processo do cliente preservado.
Cliente Vendas sem Config Server, na porta 18081 do teste	Inicia com importação opcional e exibe “Configuração não carregada”.

Escopo da validação: os testes utilizaram um repositório Git temporário local (file://), com os seis arquivos fornecidos. A publicação e o acesso ao repositório GitHub remoto precisam ser conferidos após inserir a URL real. Código, relatório, roteiro de vídeo e instruções de publicação acompanham o projeto.

Referências: docs.spring.io/spring-cloud-config/reference/client.html; spring.io/guides/gs/centralized-configuration/; spring.io/blog/2026/08/20/spring-cloud-2025-1-3-has-been-released/